

Database Management Systems

Database Transactions, ACID Properties

Dr Nagasundari S

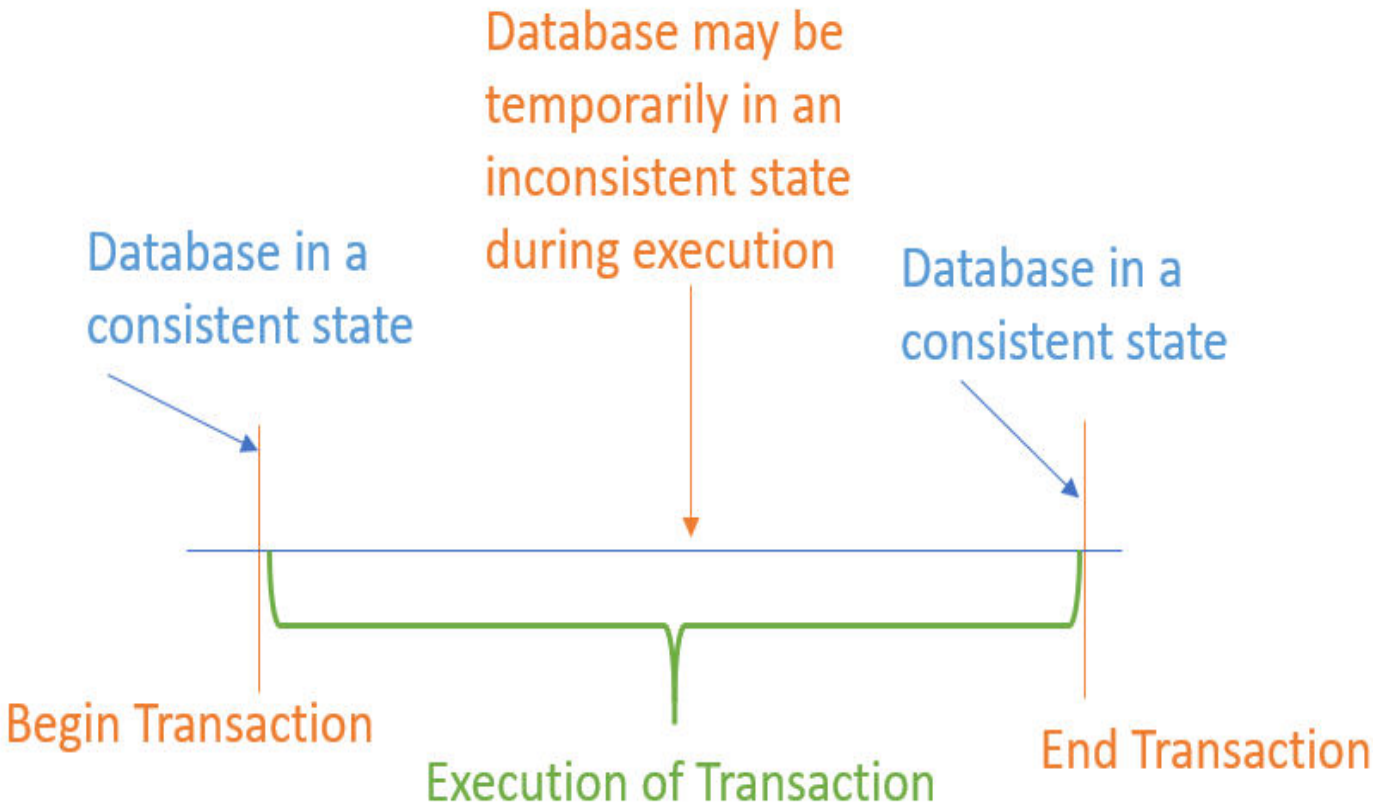
Department of Computer Science and Engineering

Anchor: Dr Nagasundari S, Dept of CSE

Database Management Systems

Outline

- ❑Transaction Concept
- ❑Simple Transaction Model
- ❑ACID Properties



Single-User versus Multi User Systems

- One criterion for classifying a database system is according to the number of users who can use the system concurrently.

- A DBMS is a single-user system if, at most, one user at a time can use it.

- A DBMS is multi-user if many users can use the system and access the database concurrently.

Introduction to Transactions

A transaction is a unit of program execution in a database system that:

- Accesses (reads) one or more data items.
- Possibly updates (writes/changes) one or more data items.

It is the smallest unit of work that is either fully completed or fully rolled back (no partial completion is allowed).

Transaction may be initiated by a user program written in a high-level data-manipulation language (typically SQL), or programming language (e.g., C++ or Java), with embedded database accesses in JDBC or ODBC.

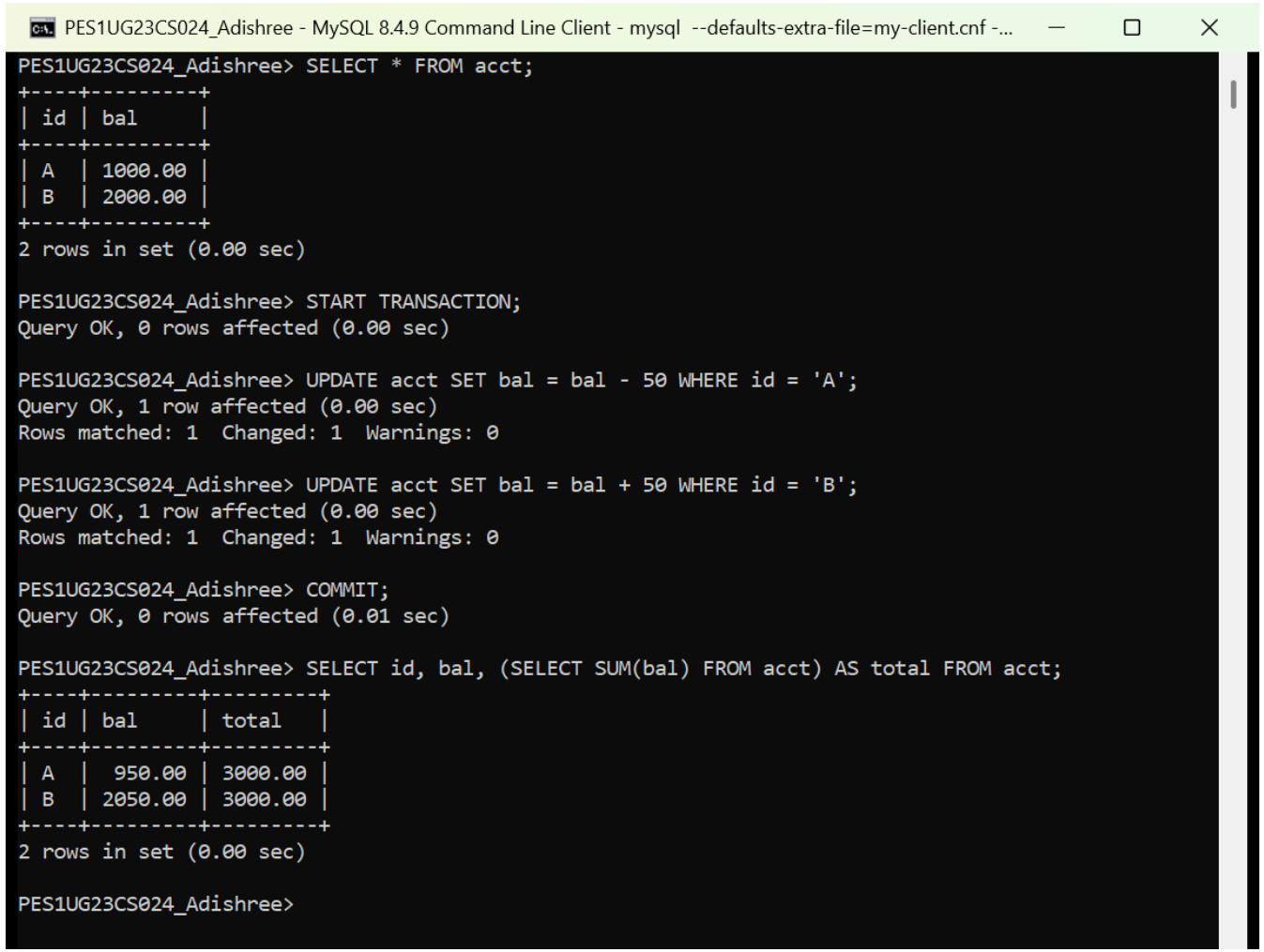
- E.g., transaction to transfer $50 from account A to account B:

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Database Management Systems

Execution Verification — Fund Transfer

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: transfer of $50 from A to B, committed as one unit (A+B stays 3000).



Introduction to Transactions

A collection of several operations on the database appears to be a single unit from the point of view of the database user. However, the database system consists of several operations.

For Example: A transfer of funds from one account to another account is a single operation from the customer's standpoint; however, within the database system, it consists of several operations.

It's essential that these operations occur, or that, in case of failure, none occur.

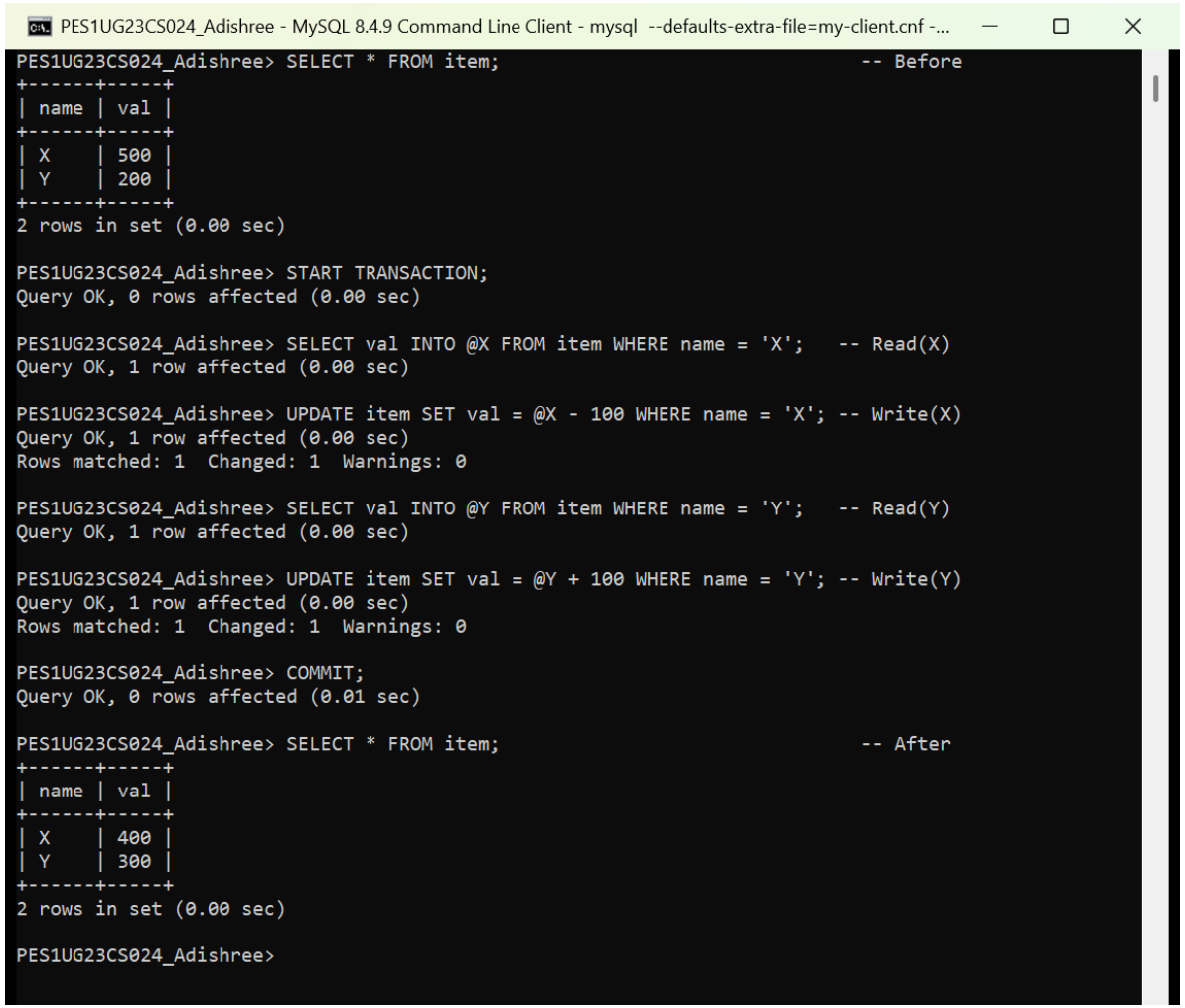
Collections of operations that form a single logical unit of work are called transactions.

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Database Management Systems

Execution Verification — Transaction T

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: Read/Write steps of transaction T on X=500, Y=200 (after: X=400, Y=300).



Transaction Concept

A transaction is delimited by statements (or function calls) of the form begin transaction and end transaction*. The transaction consists of all operations to be executed between the begin transaction and the end transaction.*

Begin and end transaction statements specify transaction boundaries, all operations between them are part of a single transaction.

- **Read-only transaction**: If the database operations in a transaction do not update the database.
- **Read-write transaction:** If the database operations in a transaction update the database as well as retrieve.

Two main issues to deal with transactions:

- **Failures of various kinds, such as hardware failures and system crashes during the transaction**
- **Concurrent execution of multiple transactions**

A Simple Transactional Model

Transactions access data using two operations:

- **read(X):** which transfers the data item X from the database to a variable, also called X, in a buffer in main memory belonging to the transaction that executed the read operation.
- **write(X):** which transfers the value in the variable X in the main-memory buffer of the transaction that executed the write to the data item X in the database.
- It is important to know if a change to a data item appears only in main memory or if it has been written to the database on disk.
- In a real database system, the write operation does not necessarily result in the immediate update of the data on the disk; the write operation may be temporarily stored elsewhere and executed on the disk later. For now, however, we shall assume that the write operation updates the database immediately.

- **Executing a read_item(X) command includes the following steps**
  1. Find the address of the disk block that contains item X.
  2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer). The size of the buffer is the same as the disk block size.
  3. Copy item X from the buffer to the program variable named X.
- **Executing a write_item(X) command includes the following steps**
  1. Find the address of the disk block that contains item X.
  2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
  3. Copy item X from the program variable named X into its correct location in the buffer.
  4. Store the updated disk block from the buffer back to disk (either immediately or at some later point in time).

Data Buffers:

- The DBMS maintains data buffers in main memory, each holding one database disk block.
- When all buffers are full, a buffer replacement policy decides which buffer to replace.
- LRU (Least Recently Used) is a common replacement policy used to choose the buffer to replace.
- If a buffer is modified, it is written back to disk before being reused.
- Some buffer replacement policies are specific to DBMS characteristics.

Transaction Concurrency

Transaction-processing systems usually allow multiple transactions to run concurrently. Allowing multiple transactions to update data concurrently causes several complications with consistency of the data, **Ensuring consistency despite concurrent execution of transactions requires extra work; it is far easier to insist that transactions run serially— that is, one at a time, each starting only after the previous one has completed**.

However, there are two good reasons for allowing concurrency:

- **Improved throughput and resource utilization**
- **Reduced waiting time**.

Transaction Execution States / State Transition diagram

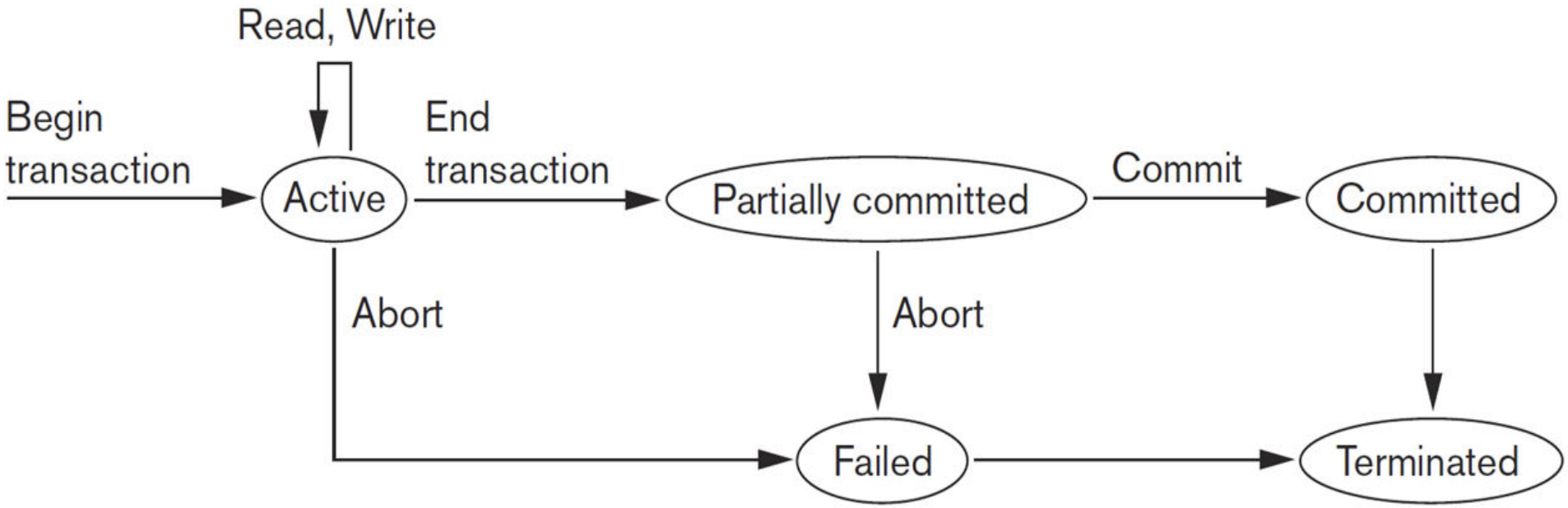


Figure 20.4
State transition diagram illustrating the states for transaction execution.

Transaction States

State transition diagram illustrates how a transaction moves through its execution states.

A transaction goes into an **active state** immediately after it starts execution, where it can execute its READ and WRITE operations.

When the transaction ends, it moves to the **partially committed state**. At this point, some types of concurrency control protocols may do additional checks to see if the transaction can be committed or not. Also, some recovery protocols need to ensure that a system failure will not result in an inability to record the changes of the transaction permanently.

If these checks are successful, the transaction is said to have reached its commit point and enters the **committed state.** When a transaction is committed, it has concluded its execution successfully and all its changes must be recorded permanently in the database, even if a system failure occurs.

Transaction States

A transaction can go to the **failed state** if one of the checks fails or if the transaction is aborted during its active state.

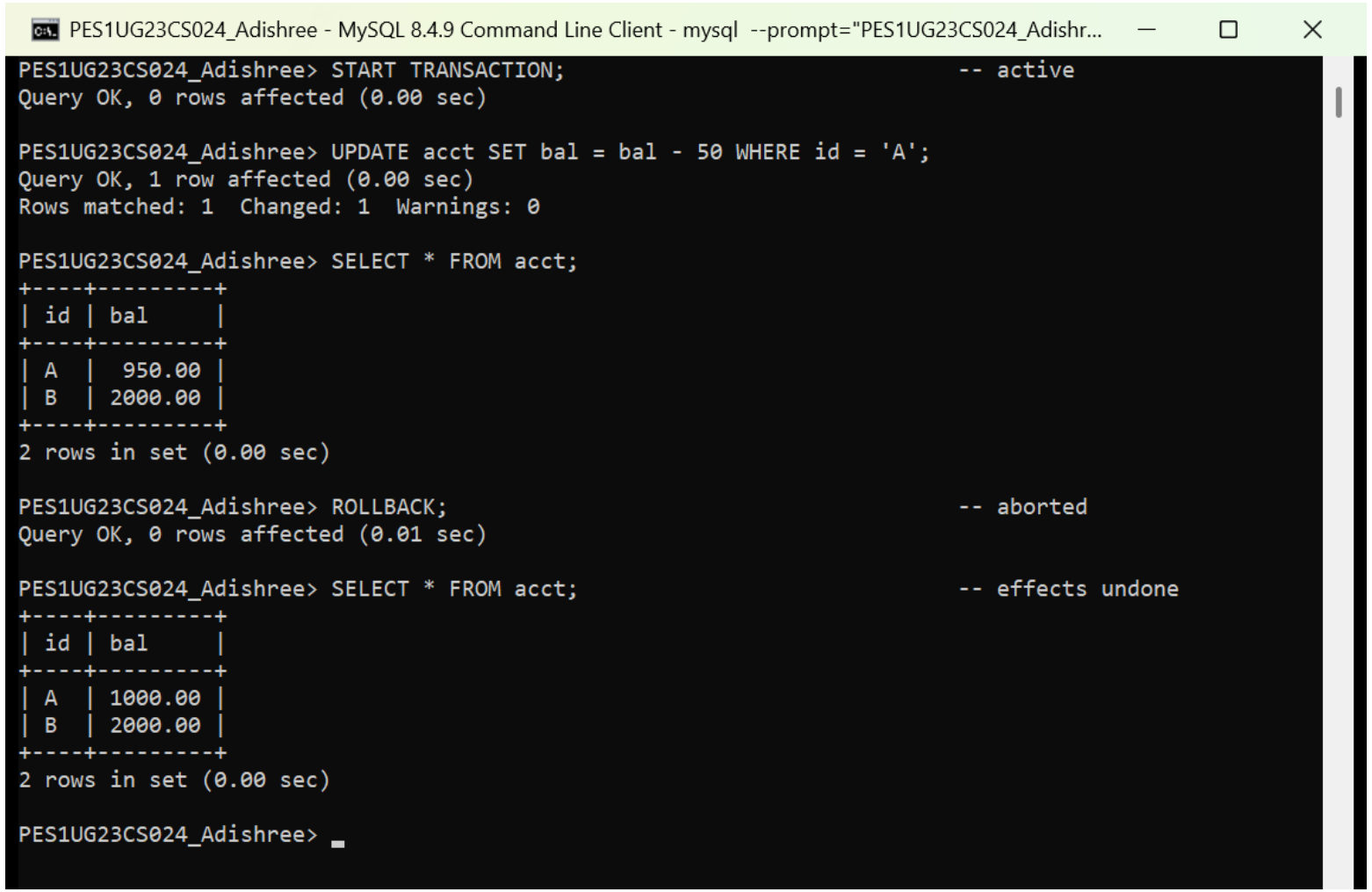
The **terminated state** corresponds to the transaction leaving the system.

Failed or aborted transactions may be restarted later—either automatically or after being resubmitted by the user—as brand new transactions.

Database Management Systems

Execution Verification — Failed / Aborted State

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: active transaction aborted with ROLLBACK; its update is undone.



Transaction States

When the transaction is aborted there are two options:

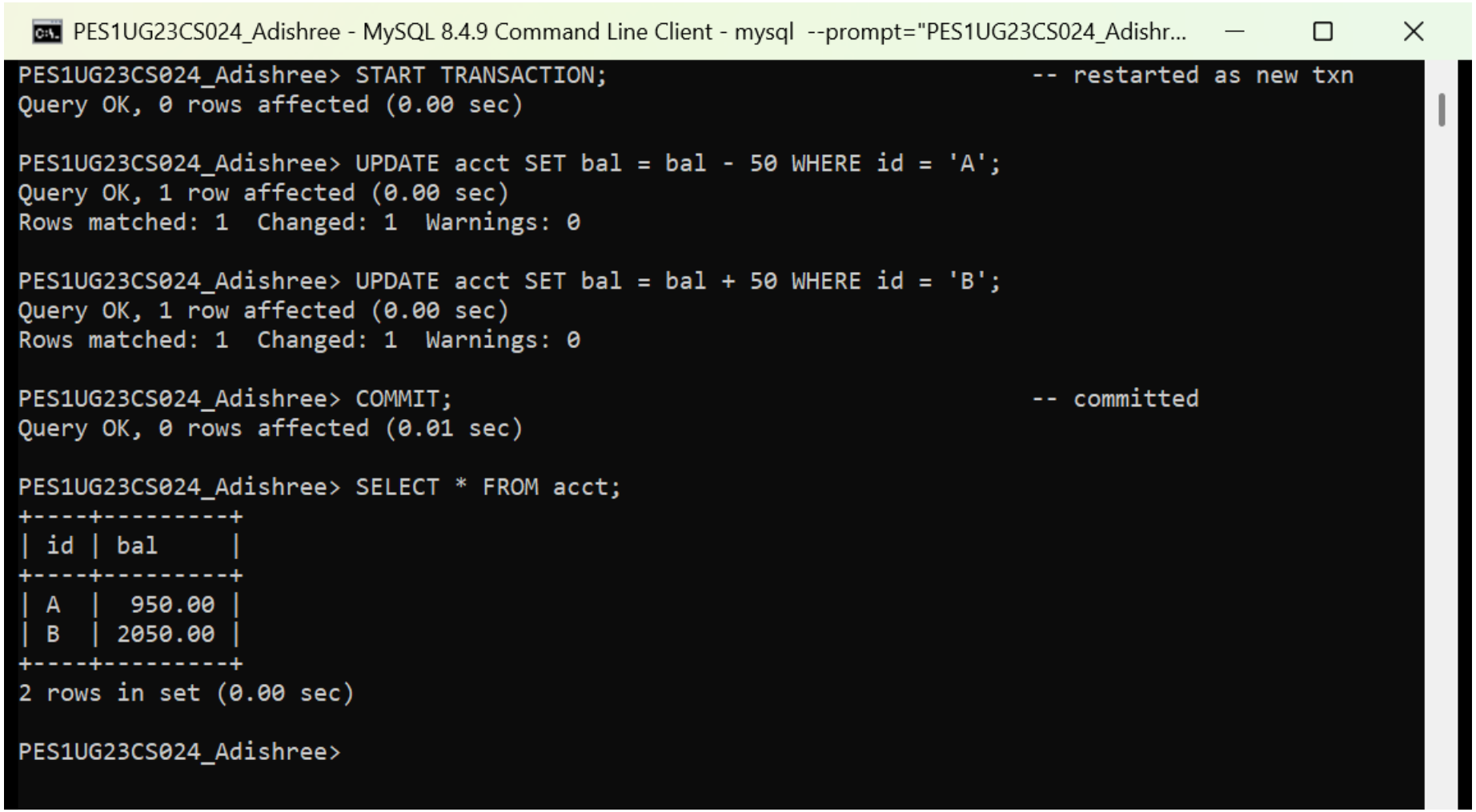
- It can **restart** the transaction, but only if the transaction was aborted as a result of some hardware or software error that was not created through the internal logic of the transaction. A restarted transaction is considered to be a new transaction.

- It can **kill** the transaction. It usually does so because of **some internal logical error that can be corrected only by rewriting the application program**, or because the input was bad, or because the desired data were not found in the database

Database Management Systems

Execution Verification — Restarted Transaction

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: the aborted work is resubmitted as a new transaction and committed.



Transaction cont…

- BEGIN_TRANSACTION. This marks the beginning of transaction execution.

- READ or WRITE. These specify read or write operations on the database items that are executed as part of a transaction.

- END_TRANSACTION. This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.

- COMMIT_TRANSACTION. This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.

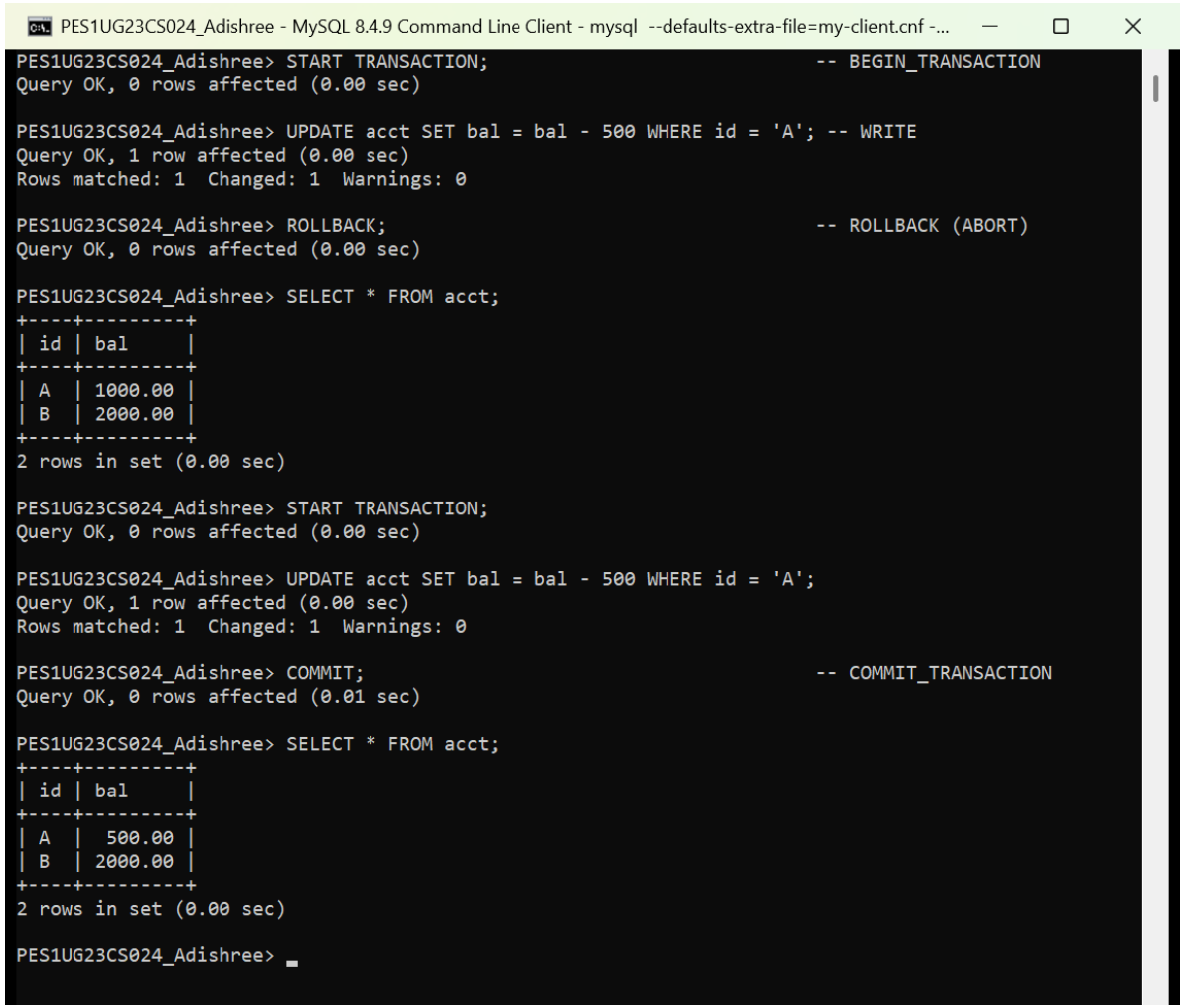
- ROLLBACK (or ABORT). This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

Database Management Systems

Execution Verification — COMMIT / ROLLBACK

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: ROLLBACK undoes the write; COMMIT makes it permanent.

PES UNIVERSITY
CELEBRATING 50 YEARS



Transaction Support in SQL

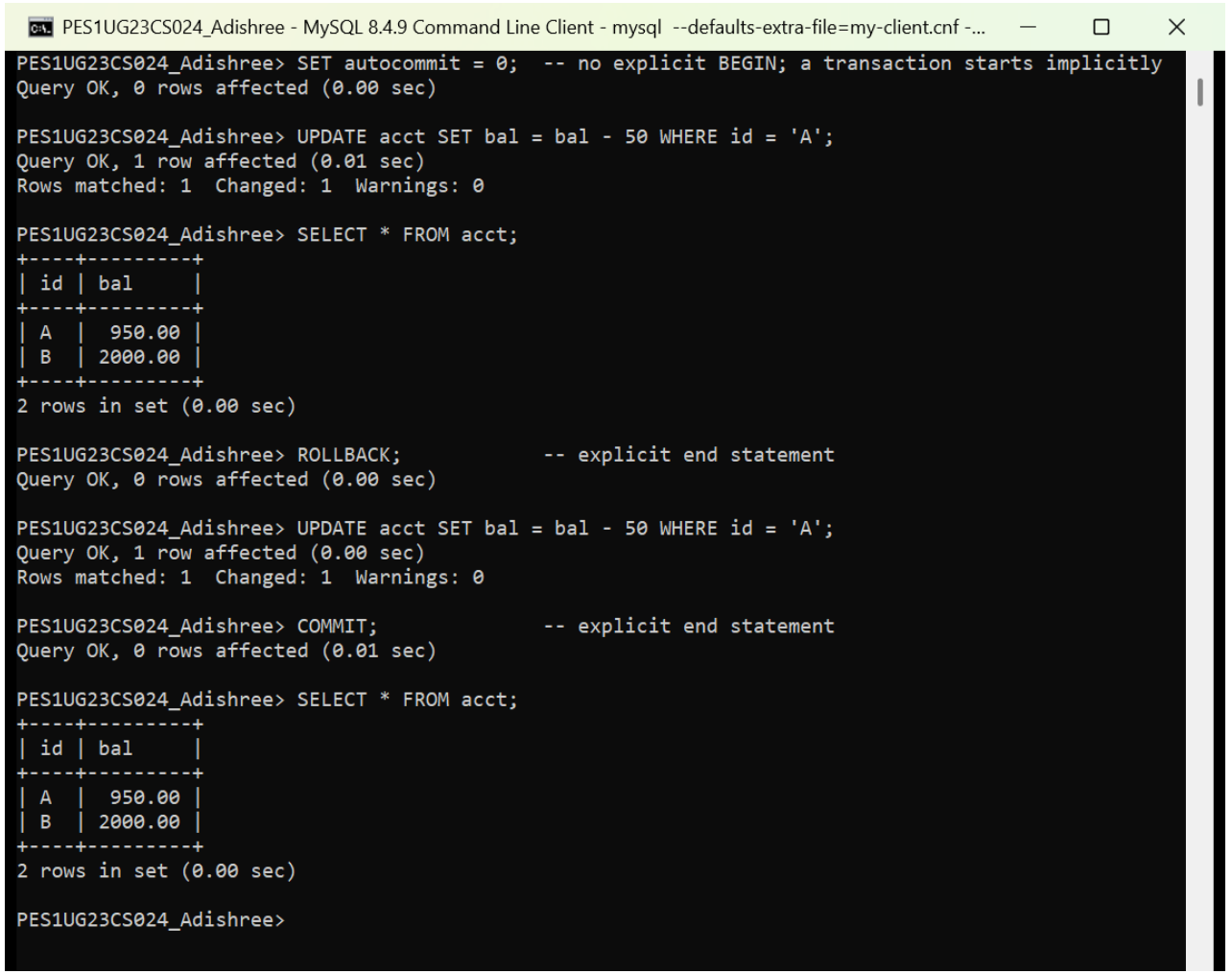
There is a lot of support from SQL for the development of transaction-based applications

- No explicit Begin Transaction statement is needed by SQL.
- Every transaction must have an explicit end statement
  - COMMIT
  - ROLLBACK
- Access mode is READ ONLY or READ WRITE

Database Management Systems

Execution Verification — Transaction Support in SQL

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: with autocommit=0 no BEGIN is needed; each transaction ends with ROLLBACK or COMMIT.



Access Mode

Access mode in SQL transactions defines the type of operations that a transaction can perform.

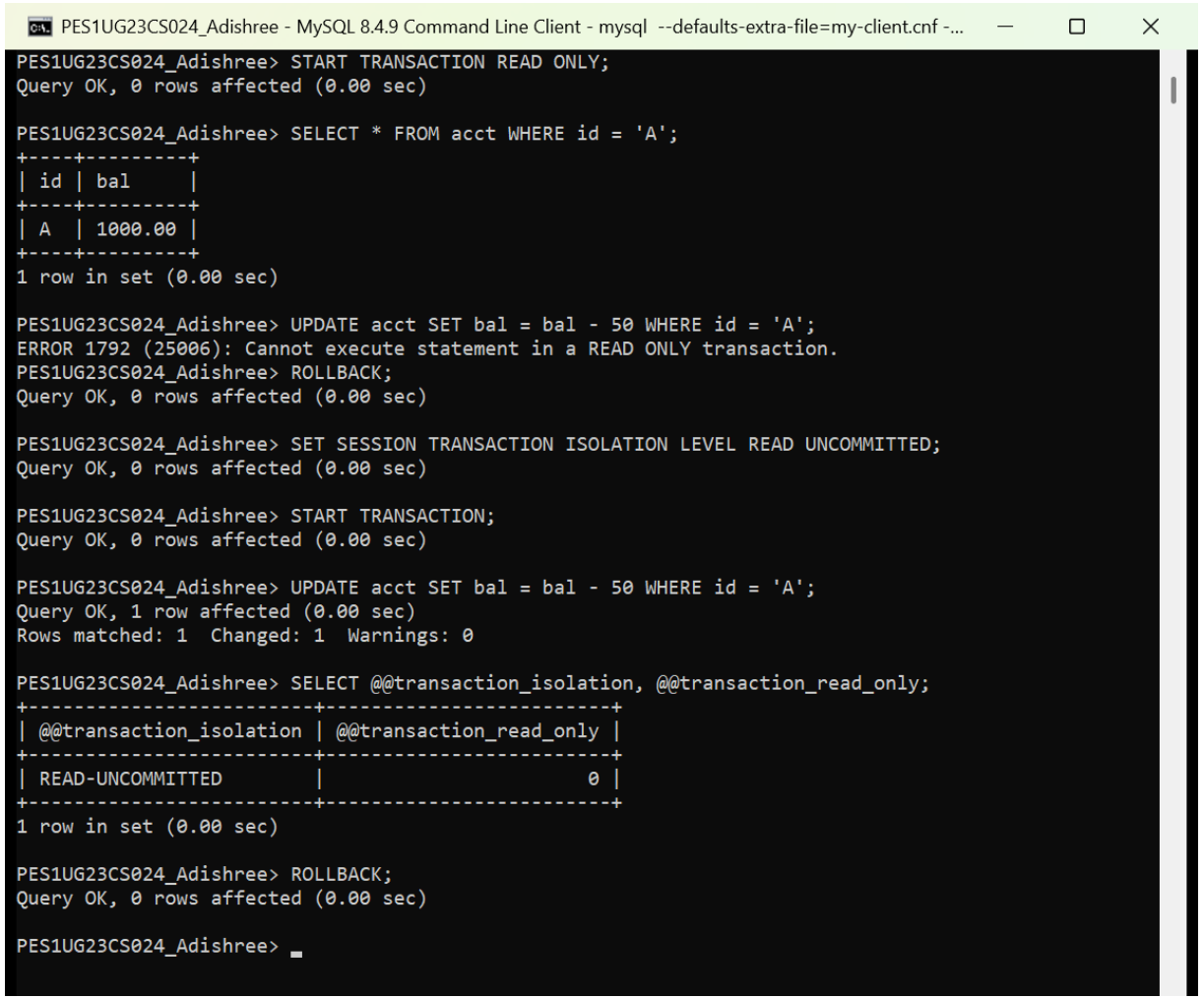
There are two primary access modes:

- **READ WRITE:**
  - Default mode. (MySQL allows writes at every isolation level, including READ UNCOMMITTED.)
  - Allows all operations: SELECT, UPDATE, INSERT, DELETE, and CREATE commands.
- **READ ONLY:**
  - Set explicitly: START TRANSACTION READ ONLY / SET TRANSACTION READ ONLY (the SQL-92 rule that READ UNCOMMITTED implies READ ONLY is not applied by MySQL).
  - Restricts transactions to data retrieval only (i.e., SELECT statements).

Database Management Systems

Execution Verification — Access Mode

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: READ ONLY rejects the UPDATE (ERROR 1792); READ UNCOMMITTED still permits writes.



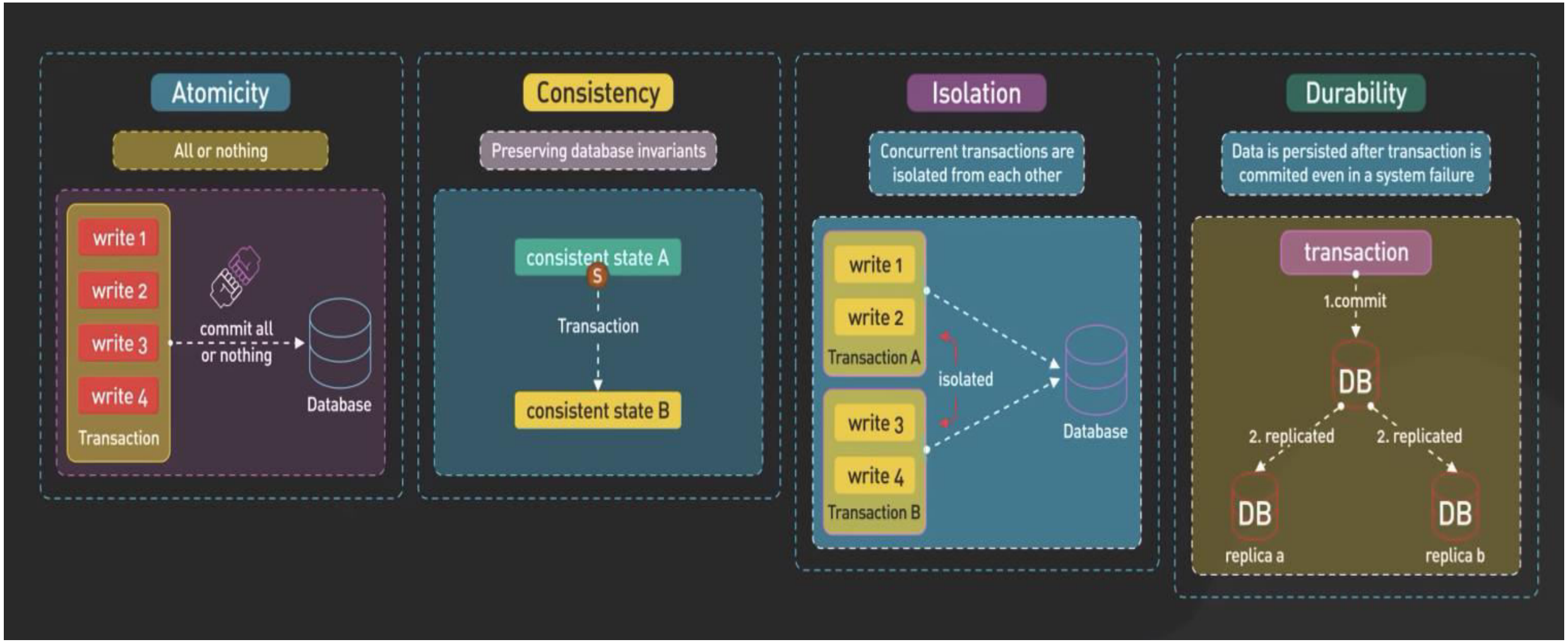
ACID Properties

To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are. This “all-or-none” property is referred to as atomicity.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. **Intermediate transaction results must be hidden from other concurrently executed transactions.**
- **Durability.** After a transaction completes successfully, the changes it has made to the database must persist, even if there are system failures.

Database Management Systems

ACID Properties



Example of Fund Transfer

We shall illustrate the transaction concept using a simple bank application consisting of several accounts and a set of transactions that access and update those accounts.

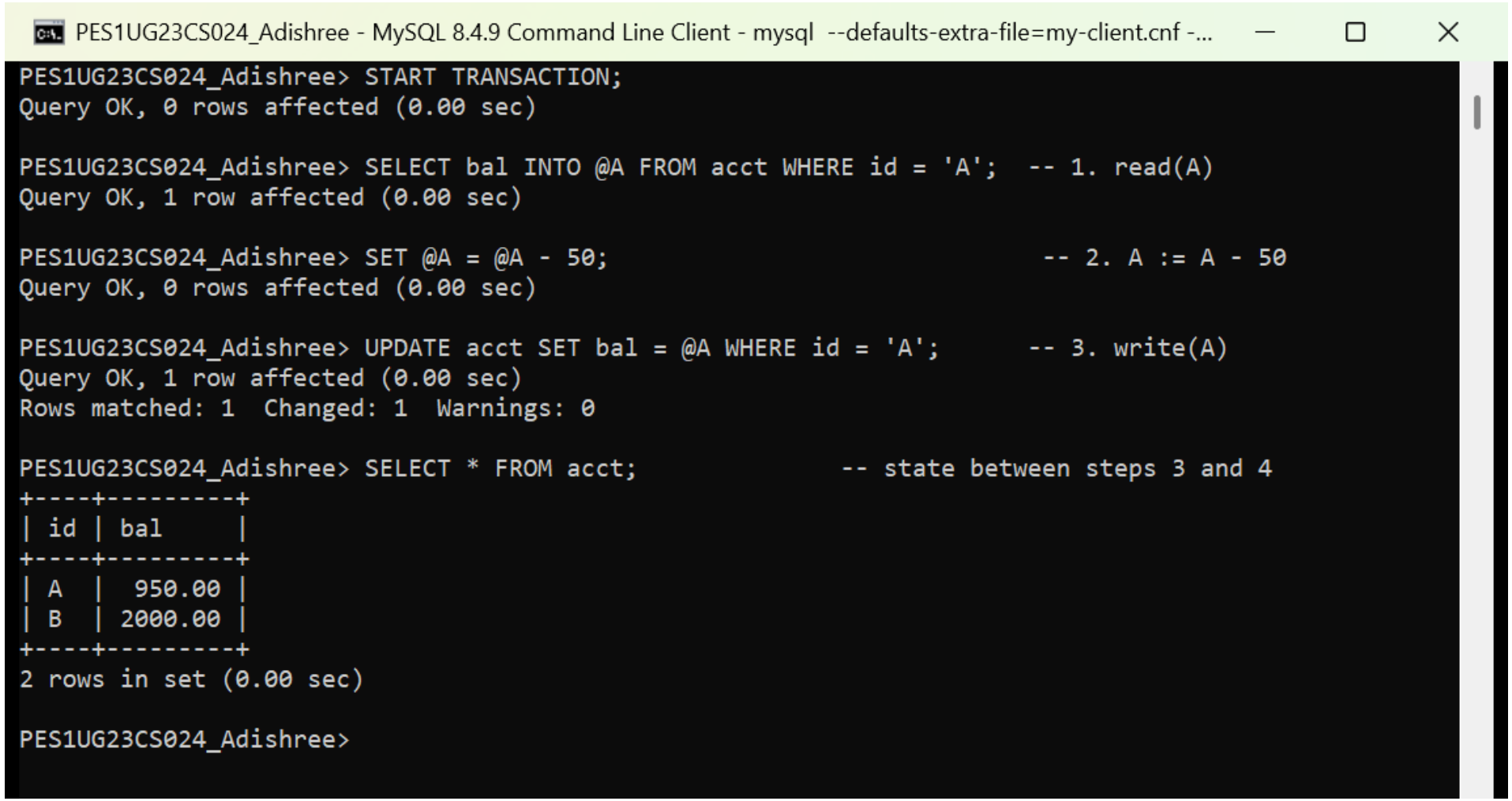
Transaction to transfer $50 from account A to account B: Transaction (Ti)

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Database Management Systems

Execution Verification — Fund Transfer Steps 1-6 (1/2)

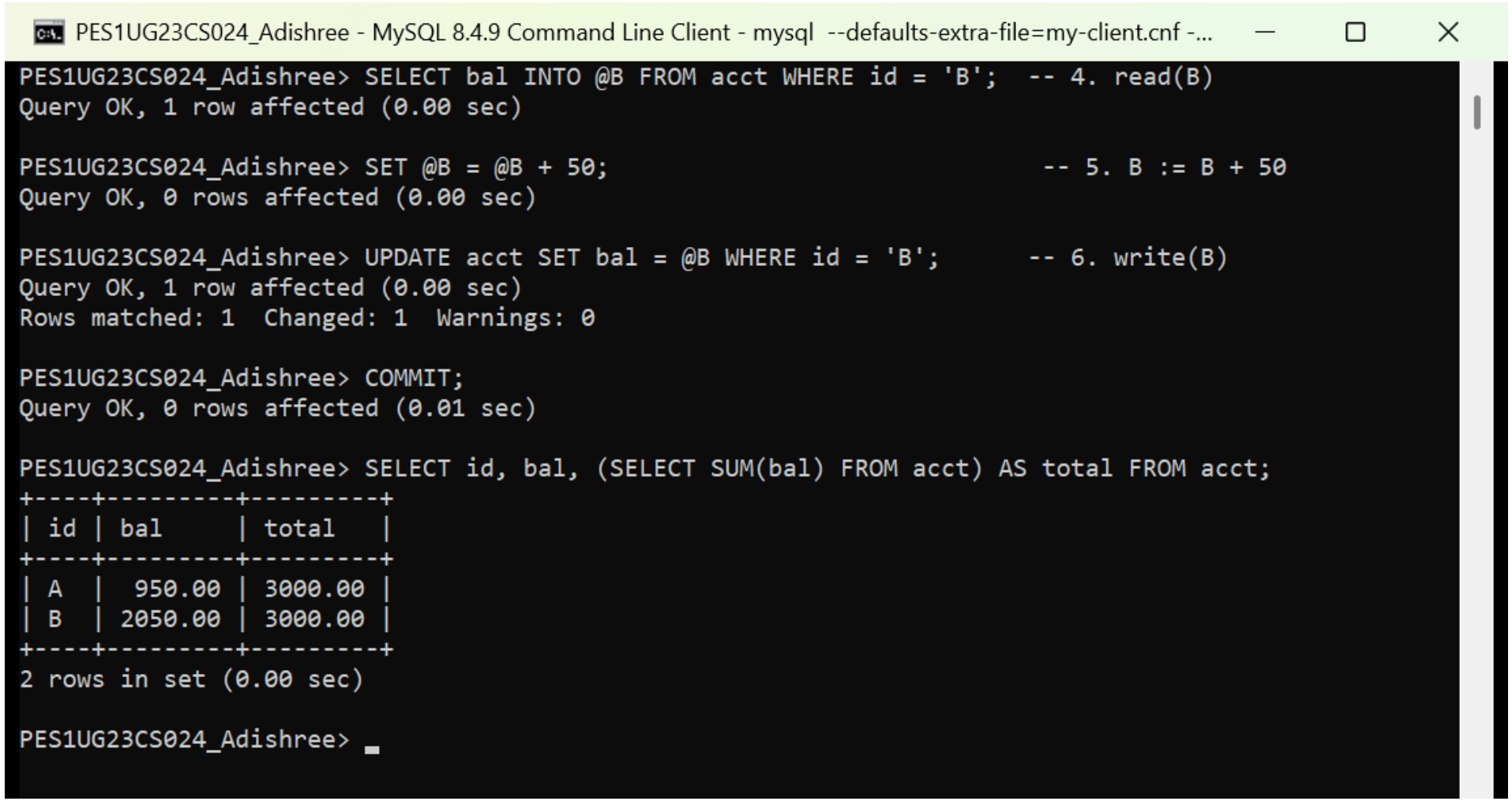
Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: steps 1-6; left = after step 3 (A=950, B=2000), right = after COMMIT.



Database Management Systems

Execution Verification — Fund Transfer Steps 1-6 (2/2)

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: steps 1-6; left = after step 3 (A=950, B=2000), right = after COMMIT.



Example of Fund Transfer

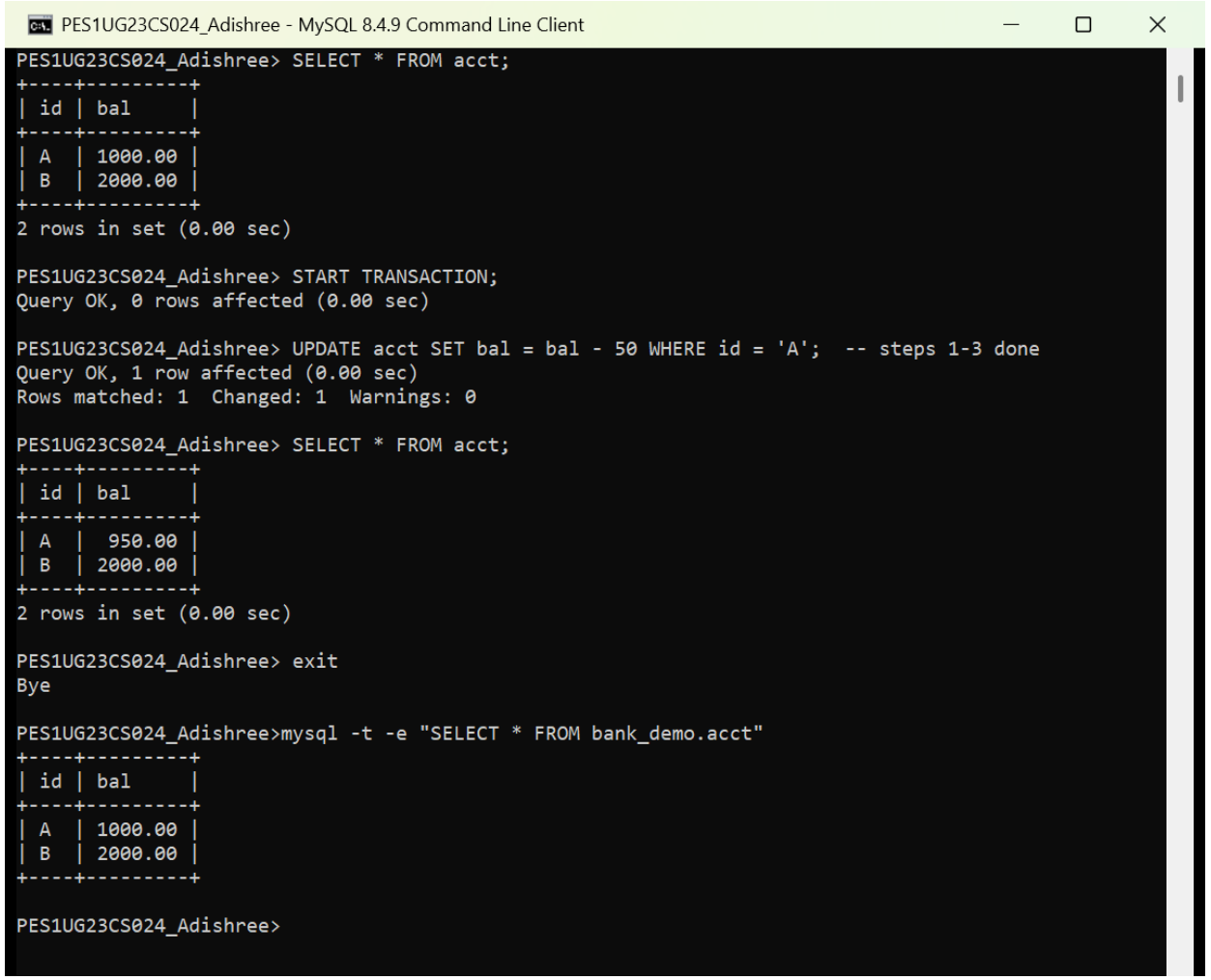
Atomicity requirement

- If the transaction fails after step 3 and before step 6, **money will be "lost" leading to an inconsistent database state**
- Failure could be due to software or hardware
- The system should ensure that *updates of a partially executed transaction are not reflected in the database.*
- If the atomicity property is present, all actions of the transaction are reflected in the database, or none are
- The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write. This information is written to a file called the log.
- If the transaction does not complete its execution, the database system restores the old values from the log to make it appear as though the transaction never executed. Ensuring atomicity is the responsibility of the database system; specifically, it is handled by a component of the database called the **recovery system.**

Database Management Systems

Execution Verification — Atomicity

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: session ends after step 3 without COMMIT; InnoDB rolls back, A and B are unchanged.



Example of Fund Transfer

- **Durability:** Once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), **the updates to the database by the transaction must persist even if there are software or hardware failures.** The recovery system of the database, is responsible for ensuring durability, in addition to ensuring atomicity.

- **Consistency**:
  - The sum of A and B is unchanged by the execution of the transaction

  In general, consistency requirements include
  - Explicitly specified integrity constraints such as primary keys and foreign keys
  - Implicit integrity constraints
  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - When the transaction completes successfully the database must be consistent

Database Management Systems

Execution Verification — Consistency and Durability

Executed live on MySQL 8.4.9 (InnoDB) as PES1UG23CS024_Adishree: CHECK(bal>=0) rule, committed transfer read back from a new session (A+B = 3000).

PES1UG23CS024_Adishree - MySQL 8.4.9 Command Line Client

```
PES1UG23CS024_Adishree> ALTER TABLE acct ADD CONSTRAINT chk_bal CHECK (bal >= 0);
Query OK, 2 rows affected (0.07 sec)
Records: 2  Duplicates: 0  Warnings: 0

PES1UG23CS024_Adishree> SELECT SUM(bal) AS "A+B before" FROM acct;
+------------+
| A+B before |
+------------+
|    3000.00 |
+------------+
1 row in set (0.00 sec)

PES1UG23CS024_Adishree> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_Adishree> UPDATE acct SET bal = bal - 50 WHERE id = 'A';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES1UG23CS024_Adishree> UPDATE acct SET bal = bal + 50 WHERE id = 'B';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES1UG23CS024_Adishree> COMMIT;
Query OK, 0 rows affected (0.02 sec)

PES1UG23CS024_Adishree> exit
Bye

PES1UG23CS024_Adishree>mysql -t -e "SELECT id, bal FROM bank_demo.acct"
+----+---------+
| id | bal     |
+----+---------+
| A  |  950.00 |
| B  | 2050.00 |
+----+---------+

PES1UG23CS024_Adishree>
```

Example of Fund Transfer

- **Isolation** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database**, it will see an inconsistent database (the sum *A + B* will be less than it should be).**

T1	T2
1. **read**(*A*)	
2. *A* := *A* – 50	
3. **write**(*A*)	
	read(A), read(B), print(A+B)
4. **read**(*B*)	
5. *B* := *B* + 50	
6. **write**(*B*)	

Isolation can be ensured trivially by running transactions serially. That is, one after the other. However, executing multiple transactions concurrently has significant benefits.

Database Management Systems

Execution Verification — Isolation

Two concurrent MySQL 8.4.9 sessions of PES1UG23CS024_Adishree (T1/T2 in the prompt): READ UNCOMMITTED sees A+B = 2950 mid-transfer; READ COMMITTED sees 3000.

PES1UG23CS024_Adishree - MySQL 8.4.9 Command Line Client - T1 - mysql --prompt="PES1UG...

```
PES1UG23CS024_Adishree [T1]> START TRANSACTION;                              -- step 1
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_Adishree [T1]> UPDATE acct SET bal = bal - 50 WHERE id = 'A'; -- step 2
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES1UG23CS024_Adishree [T1]> UPDATE acct SET bal = bal + 50 WHERE id = 'B'; -- step 5
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES1UG23CS024_Adishree [T1]> COMMIT;                                         -- step 6
Query OK, 0 rows affected (0.01 sec)

PES1UG23CS024_Adishree [T1]>
```

PES1UG23CS024_Adishree - MySQL 8.4.9 Command Line Client - T2 - mysql --prompt="PES1UG...

```
PES1UG23CS024_Adishree [T2]> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_Adishree [T2]> SELECT SUM(bal) AS "A+B" FROM acct;  -- step 3
+---------+
| A+B     |
+---------+
| 2950.00 |
+---------+
1 row in set (0.00 sec)

PES1UG23CS024_Adishree [T2]> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_Adishree [T2]> SELECT SUM(bal) AS "A+B" FROM acct;  -- step 4
+---------+
| A+B     |
+---------+
| 3000.00 |
+---------+
1 row in set (0.00 sec)

PES1UG23CS024_Adishree [T2]> SELECT SUM(bal) AS "A+B" FROM acct;  -- step 7
+---------+
| A+B     |
+---------+
| 3000.00 |
+---------+
1 row in set (0.00 sec)

PES1UG23CS024_Adishree [T2]>
```

Example of Fund Transfer

- Concurrent execution of transactions provides significant performance benefits.
- **The isolation property of a transaction ensures that the concurrent execution of transactions results in a system state that is equivalent to a state that could have been obtained had these transactions executed one at a time in some order**.
- Ensuring the isolation property is the responsibility of a component of the database system called the **concurrency-control system**.

Q&A

1. Which of the following ACID properties ensures that once a transaction completes, the changes made will persist even if the system crashes afterward?

(a) Atomicity.
(b) Consistency.
(c) Isolation.
(d) Durability.

2. In a transaction that transfers $50 from account A to account B, what happens if the system fails after updating account A but before updating account B?

(a) The system remains consistent.
(b) The transaction is completed successfully.
(c) The system enters an inconsistent state.
(d) The transaction automatically retries.

37

Q&A

3. Which ACID property ensures that a transaction, when executed in isolation, appears as if no other transactions are executing simultaneously?

(a) Atomicity
(b) Consistency
(c) Isolation
(d) Durability

4. Which of the following best describes a transaction in a database system?

(a) A transaction is a set of operations that only reads data from the database.
(b) A transaction is a unit of program execution that consists of a single operation.
(c) A transaction is a collection of operations that either all occur or none occur, ensuring data integrity.
(d) A transaction always updates the database immediately after an operation is executed.

Q&A

1. Which of the following ACID properties ensures that once a transaction completes, the changes made will persist even if the system crashes afterward?

(a) Atomicity.
(b) Consistency.
(c) Isolation.
(d) Durability.

2. In a transaction that transfers $50 from account A to account B, what happens if the system fails after updating account A but before updating account B?

(a) The system remains consistent.
(b) The transaction is completed successfully.
(c) The system enters an inconsistent state.
(d) The transaction automatically retries.

Q&A

3. Which ACID property ensures that a transaction, when executed in isolation, appears as if no other transactions are executing simultaneously?

(a) Atomicity
(b) Consistency
(c) Isolation
(d) Durability

4. Which of the following best describes a transaction in a database system?

(a) A transaction is a set of operations that only reads data from the database.
(b) A transaction is a unit of program execution that consists of a single operation.
(c) A transaction is a collection of operations that either all occur or none occur, ensuring data integrity.
(d) A transaction always updates the database immediately after an operation is executed.



THANK YOU

Dr Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu